

Brief Insights

1. Data Cleaning

The dataset contained duplicate records and missing values. Duplicate rows were removed using `dropDuplicates()`, while null values in age, salary, department, and region columns were replaced with appropriate default values using `fillna()`. This improved data quality and consistency.

2. Data Transformation

Several transformations were applied to prepare the data for analysis. The department column was renamed to improve readability, and the salary column was cast to double type for accurate numerical computations. These transformations demonstrate the immutability of Spark DataFrames, where each operation creates a new DataFrame.

3. Data Filtering

Filtering operations were performed to extract specific subsets of data, such as employees aged between 25 and 40, employees belonging to the IT department, and employees from the North region. This helped focus analysis on relevant records.

4. Aggregation Analysis

Aggregation functions such as `count()`, `sum()`, `avg()`, `min()`, and `max()` were used to generate statistical insights from the dataset. These functions provided information about employee count, salary distribution, and overall workforce characteristics.

5. Grouping and Summary Statistics

Using `groupBy()`, department-wise summaries were generated. This revealed the number of employees and average salary in each department, making it easier to compare departmental performance and workforce distribution.

6. Wide Transformations and Shuffle

The `groupBy()` operation is a wide transformation because data must be shuffled across partitions to group records with the same department value. Shuffle operations are generally more expensive than narrow transformations such as `filter()` and `withColumn()`.

7. End-to-End Pipeline

A complete Spark pipeline was built by combining data cleaning, transformation, filtering, and aggregation steps. This demonstrates how Spark DataFrames can be used to process large datasets efficiently in a structured and scalable manner.

Conclusion

Apache Spark provides fast and efficient distributed data processing through in-memory computation. By cleaning inconsistent data, applying transformations, and performing aggregations, meaningful insights can be extracted from raw datasets while maintaining scalability and performance.